

CS471 – Quiz 8 Study Guide

Lessons 28–31: Neural Networks & Deep Learning

Study Strategy

This study guide synthesizes the core architectures and training mechanisms of Artificial Neural Networks and CNNs. For Quiz 8, focus heavily on understanding the *training loop* (how error moves backward), the mathematical purpose of *hyperparameters* (like learning rate), and *why* CNNs are structurally superior to standard networks for imagery.

Lessons 28 & 29: Neural Network Architecture & Training

The Anatomy of a Neural Network

- **Weights:** Scale the input signals. They determine the mathematical "importance" of a specific connection between two neurons.
- **Biases:** Shift the activation threshold left or right, allowing the neuron to trigger even if all input signals are zero.
- **Activation Functions (e.g., ReLU, Sigmoid):** Crucial mathematical gates that introduce **non-linearity** into the network. Without them, the entire neural network would collapse into a single, flat linear regression model incapable of solving complex, curved tactical boundaries.

The Training Loop: How a Network "Learns"

Training is an iterative cycle of making predictions, measuring errors, and updating weights. One complete pass of the entire training dataset is called an **Epoch**.

1. **Forward Pass:** Input data flows through the network layers to produce a final prediction.
2. **Loss Function:** Mathematically quantifies exactly how wrong the prediction was compared to the true label.
3. **Backpropagation:** The algorithm works backward from the output layer, using the chain rule of calculus to compute the gradient (slope of the error curve) for every single weight and bias in the network.
4. **Gradient Descent:** Uses those calculated gradients to step the weights "downhill" toward a lower error state for the next forward pass.

The Learning Rate

The learning rate dictates the mathematical step size taken during Gradient Descent.

- **Too High:** The network takes massive steps, violently overshooting the minimum error. Training becomes erratic, unstable, and fails to converge.
- **Too Low:** Training becomes painfully slow, and the model risks getting permanently stuck in a sub-optimal local minimum (a shallow valley on the error curve).

Lesson 30: Deep Learning

Hierarchical Representation

”Deep” networks utilize multiple hidden layers to build understanding sequentially.

- **Early Layers:** Extract very simple, low-level features (e.g., straight edges, shadows, contrast).
- **Middle Layers:** Combine those basic edges into structural shapes (e.g., tank treads, wing swept-angles).
- **Final Layers:** Combine those shapes into highly complex, high-level object representations (e.g., classifying the object as an adversary fighter jet) before the final output.

The Vanishing Gradient Problem

Backpropagation multiplies gradients backward through the layers. If these gradients are smaller than 1.0, multiplying them repeatedly causes the error signal to shrink exponentially. By the time the signal reaches the earliest layers, the gradient is so close to zero that the weights barely update.
Result: The foundational layers of very deep networks never effectively learn.

Dropout (Regularization)

A technique to prevent severe overfitting (High Variance). During training, Dropout **randomly deactivates a percentage of neurons** on each pass. This prevents the network from relying on a few dominant nodes and forces it to build diverse, redundant, and robust pathways to solve the problem.

Lesson 31: Convolutional Neural Networks

Standard ”Dense” (Fully Connected) networks connect every single pixel to every hidden node. This destroys the 2D spatial geometry of an image and causes the number of trainable parameters to explode, making them incredibly inefficient for satellite or ISR imagery.

Core CNN Components

- **Convolutional Layers & Filters (Kernels):** Small grids of weights (e.g., 3×3) that sweep across the 2D image matrix, performing dot products to extract a **Feature Map** (highlighting edges, textures, etc.). Because the exact same filter is swept across the entire image, CNNs drastically reduce the number of parameters through **weight sharing**.

- **Translation Invariance:** Because the filter sweeps everywhere, it will trigger regardless of where the target is located in the frame. A CNN can recognize a tank in the top-left corner just as easily as in the bottom-right corner.
- **Max Pooling:** Follows the convolutional layer. It sweeps across the feature map and keeps only the maximum value in each sector, discarding the rest. This **down-samples (shrinks) the spatial dimensions** of the data, dramatically reducing computational load and noise while retaining the strongest tactical signals.
- **Flattening:** Before the extracted feature maps can be classified, they must be "flattened" from 2D matrices into a 1D vector array so they can be fed into a final, standard Dense layer to output the final probability distribution.

Key Concepts Reference

1. **Activation Function:** Introduces non-linearity to solve complex boundaries.
2. **Backpropagation:** Calculates the gradient of the error backwards through the network.
3. **Learning Rate:** Controls the step size during Gradient Descent.
4. **Vanishing Gradient:** Gradients shrink exponentially, stopping early layers from learning.
5. **Translation Invariance:** The ability of a CNN filter to find a feature anywhere in an image.